

**INSTITUTO
FEDERAL**

Santa Catarina

Câmpus
São José

Laboratório 5 - Programação com Threads

Sistemas Operacionais

Bernardo Souza Muniz

31 de Março de 2026

Engenharia de Telecomunicações - IFSC-SJ

Sumário

1. Experimento 1	3
2. Experimento 2	3
3. Experimento 3	4
4. Experimento 4	5
4.1. Código 1	5
4.2. Código 2	5
5. Experimento 5	7
6. Experimento 6	8
7. Experimento 7	8
8. Experimento 8	9
9. Experimento 9	10
10. Experimento 10	11

1. Experimento 1

Ao executar o comando `ps -fgl`, foi obtido o seguinte resultado:

Figura 1: Elaborada pelo Autor

```
aluno: ~$ ps -fgl
```

F	UID	PID	PPID	PRI	NI	VSZ	RSS	WCHAN	STAT	TTY	TIME	COMMAND
0	1001	27390	18755	20	0	11764	8688	do_wai	Ss	pts/2	0:00	bash
4	1001	27565	27390	20	0	12560	5468	-	R+	pts/2	0:00	_ ps -fgl
0	1001	18782	18755	20	0	13716	10048	do_wai	Ss	pts/0	0:00	bash
0	1001	27558	18782	20	0	2468	904	hrttime	S+	pts/0	0:00	_ ./experimento1
1	1001	27559	27558	20	0	2468	96	hrttime	S+	pts/0	0:00	_ ./experimento1

```
aluno: ~$
```

Retorno do comando `ps -fgl`

É possível observar que o processo filho tem o PID 27559. Se observamos na aba deste processo ele tem o PPID 27558, que é o mesmo PID do processo acima. Ou seja, neste caso o processo pai é o 27559 e o filho 27558.

2. Experimento 2

Ao executar o comando `ps -fgl`, foi obtido o seguinte resultado:

Figura 2: Elaborada pelo Autor

```
aluno: ~$ ps -fgl
```

F	UID	PID	PPID	PRI	NI	VSZ	RSS	WCHAN	STAT	TTY	TIME	COMMAND
0	1001	27390	18755	20	0	11764	8688	do_wai	Ss	pts/2	0:00	bash
4	1001	27435	27390	20	0	12560	5536	-	R+	pts/2	0:00	_ ps -fgl
0	1001	21995	18755	20	0	11764	8504	do_sel	Ss+	pts/1	0:00	bash
0	1001	18782	18755	20	0	13716	10048	do_wai	Ss	pts/0	0:00	bash
0	1001	27381	18782	20	0	2468	900	hrttime	S+	pts/0	0:00	_ ./experimento2
0	1001	27382	27381	20	0	0	0	-	Z+	pts/0	0:00	_ [ls] <defunct>

```
aluno: ~$
```

Retorno do comando `ps -fgl`

- O processo que executou o comando `ls` foi o processo com PID 27382. No código do programa é possível observar que se a chamada `fork()` for executada (processo filho criado), o programa executa o `execve` com o comando `ls`. Além disso, no retorno do comando da figura 2, o PID do processo filho tem uma flag `[ls] <defunct>`, indicando que o processo filho ainda não terminou de executar a listagem de arquivos no diretório atual.
- No código original, é feito um `fork()`, o processo filho é criado e antes de acontecer o `execve()`, é mostrado na tela o PID do processo pai, o código `execve()` é executado e após isso é feita uma espera de 30 segundos com a função `sleep()`.
- Em comparação com o experimento 1, neste é possível ver que o PID dos processos pai e filho mudam e o código novo faz a chamada da função `execve()`.

3. Experimento 3

Ao executar o comando `pstree | grep multi_fork` e `pstree -p | grep multi_fork`, foi obtido o seguinte resultado:

Figura 3: Elaborada pelo Autor

```
aluno: ~$ pstree | grep multi_fork
|      |-gnome-terminal---+-bash--multi_fork-+-multi_fork-+-multi_fork---multi_fork
|      |                   |                  |               `--multi_fork
|      |                   |                  |               -multi_fork---multi_fork
|      |                   |                  |               `--multi_fork
|      |                   |
```

Retorno do comando `pstree | grep multi_fork`

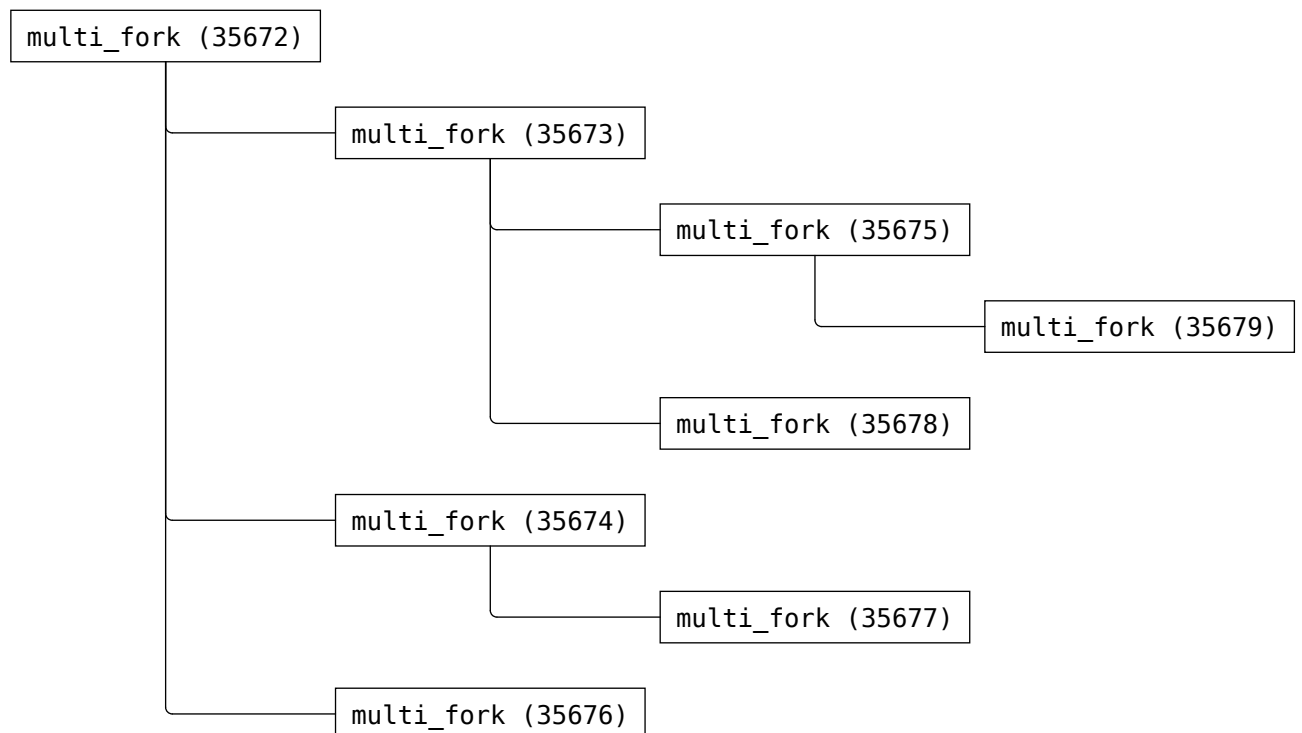
Figura 4: Elaborada pelo Autor

```
aluno: ~$ pstree -p | grep multi_fork
```

```
|      |-gnome-terminal-(18755)-+-bash(18782)---multi_fork(35672)+-multi_fork(35673)+-multi_fork(35675)---mul+  
|      |                           |                                     |  
|      |                           |                                     `--multi_fork(35678)  
|      |                           |  
|      |                           |       +-multi_fork(35674)---multi_fork(35677)  
|      |                           |       `--multi_fork(35676)
```

Retorno do comando `pstree -p | grep multi_fork`

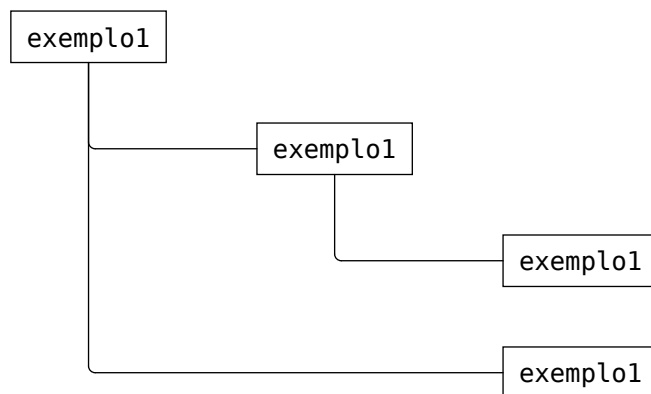
- Foram criados ao todo 8 processos.
- Porque a cada fork é gerado um processo pai e um processo filho. Como são 3 laços de repetição, cada laço cria mais um pai e mais um filho, pois cada processo gerado executa o fork, criando uma ordem de 2^3 .
- A árvore de processo pode ser visualizada da seguinte maneira:



4. Experimento 4

4.1. Código 1

- Seguindo a mesma lógica do código anterior, serão gerados 4 processos. Como temos 2 forks, o trecho de código segue a ordem de 2^n , sendo n o número de processos duplicados.
- Será exibida 4 vezes. Cada um dos processos duplicados executa a linha da mensagem de “Oi”.
- São 4 processos, no primeiro fork são gerados dois processos e estes quando executam novamente o segundo fork, geram mais dois filhos. Temos a seguinte árvore de processos:



Executando o código, foi possível observar que as expectativas foram atingidas. O código gerou quatro processos e cada um deles imprimiu uma mensagem de “Oi”

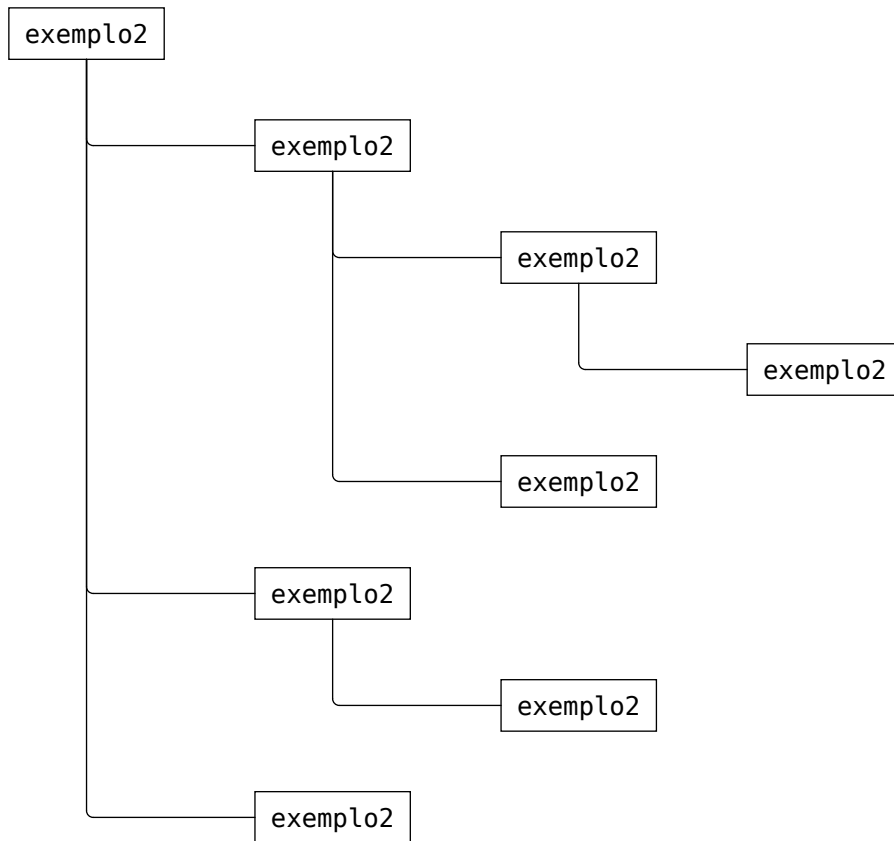
Figura 5: Elaborada pelo Autor

```
bernardo@debian:~/sop_lab5$ nano exemplo1.c
bernardo@debian:~/sop_lab5$ gcc exemplo1.c -o exemplo1
bernardo@debian:~/sop_lab5$ ./exemplo1
Oi
Oi
Oi
Oi
```

Execução do código do exemplo 4

4.2. Código 2

- Seguindo a mesma lógica do código anterior, serão gerados 8 processos. Como temos 3 forks, o trecho de código segue a ordem de 2^n , sendo n o número de processos duplicados.
- Será exibido 8 vezes. Cada um dos processos duplicados executa a linha da mensagem “Oi”
- São 8 processos, no primeiro fork são gerados dois processos e estes quando executam novamente o segundo fork, geram mais dois filhos. Temos a seguinte árvore de processos:



Executando o código, foi possível observar que as expectativas foram atingidas. O código gerou oito processos e cada um deles imprimiu uma mensagem de “Oi”

Figura 6: Elaborada pelo Autor

```
bernardo@debian:~/sop_lab5$ ./exemplo2
Oi
Oi
Oi
Oi
Oi
Oi
Oi
Oi
Oi
```

Execução do código do exemplo 4

- O crescimento exponencial pode ser explicado da seguinte maneira: Quando fazemos um único fork, dois processos são gerados (o pai, que já existia, e o filho, que foi duplicado). Se realizarmos mais um fork, o pai original cria mais um filho e o filho original cria outro filho. Conforme mais forks vão sendo executados, o código é executado tanto pelo pai e filho original, quanto pelos seus filhos.

5. Experimento 5

1) Quem encaminha a última mensagem é o processo pai. Isso acontece porque a chamada `wait()` faz com que o processo pai entre em processo de espera e volte a executar somente quanto seu filho foi encerrado.

Figura 7: Elaborada pelo Autor

```
bernardo@debian:~/sop_lab5$ ./lab_wait
[PAI]: Meu filho (PID 6073) est´a trabalhando. Vou aguardar...
[FILHO]: Iniciando tarefa pesada (5 segundos)...
[FILHO]: Tarefa conclu´ida. Vou encerrar agora com status 42.
[PAI]: Meu filho terminou normalmente com status: 42
[PAI]: Agora que meu filho encerrou, eu tamb´em vou. Tchau!
```

Execução do código do exemplo 5 com `wait()`

2) Não, o processo pai terminou antes do filho e o filho ficou imprimindo mensagens depois que o pai havia encerrado.

Figura 8: Elaborada pelo Autor

```
bernardo@debian:~/sop_lab5$ ./lab_wait
[PAI]: Meu filho (PID 6242) est´a trabalhando. Vou aguardar...
[PAI]: Meu filho terminou normalmente com status: 0
[PAI]: Agora que meu filho encerrou, eu tamb´em vou. Tchau!
[FILHO]: Iniciando tarefa pesada (5 segundos)...
bernardo@debian:~/sop_lab5$ [FILHO]: Tarefa conclu´ida. Vou encerrar agora com status 42.
□
```

Execução do código do exemplo 5 sem `wait()`

3) **WIFEXITED** verifica se o filho encerrou normalmente. Ele utiliza uma chamada com `return` ou `exit()` retornando um valor maior que zero. No exemplo, depois que o código do filho foi executado, foi feito um `exit(42)` para verificar se o filho encerrou.

WEXITSTATUS guarda o código de retorno do processo filho. No exemplo, o filho usou `exit(42)`. O macro **WEXITSTATUS** guarda o valor 42 para que o processo pai consiga imprimi-lo.

4) Um processo zumbi é um filho que terminou, porém ainda não desapareceu do sistema completamente. Se usarmos o comando `htop` veremos o estado do processo como zumbi, guardando seu PID e ocupando um lugar na tabela de processos. A chamada `wait()` entrega o status do filho ao pai. Como o pai sabe que o filho terminou e que não é mais necessário a existência do processo, a tabela de processos é liberada.

6. Experimento 6

- Listando as threads que foram criadas.

Figura 9: Elaborada pelo Autor

```
bernardo@debian:~/sop_lab5$ ps -eL | grep exp01
6714      6714 pts/0    00:00:00 exp01
6714      6715 pts/0    00:00:00 exp01
```

Execução do comando `ps -eL | grep exp01`

- Listando os arquivos que o processo abriu.

Figura 10: Elaborada pelo Autor

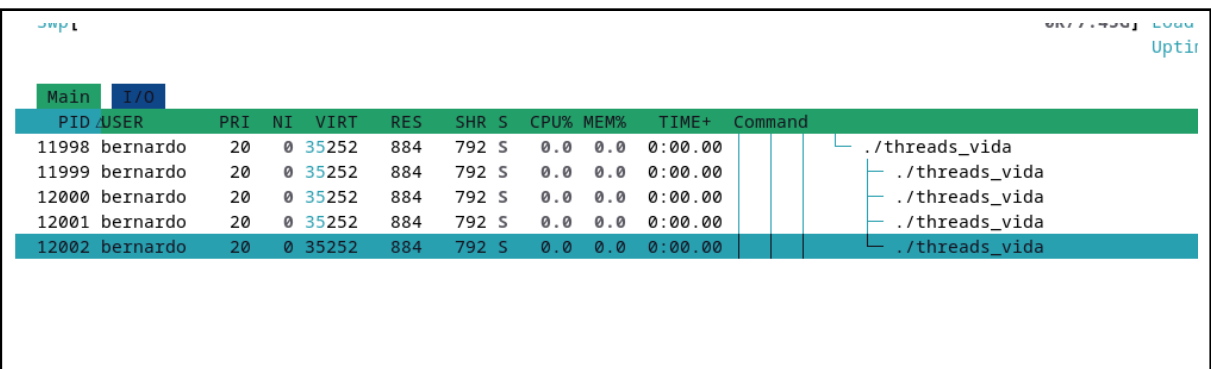
```
bernardo@debian:~/sop_lab5$ lsof -p 6714
COMMAND PID  USER  FD  TYPE DEVICE SIZE/OFF  NODE NAME
exp01   6714 bernardo cwd   DIR  259,5    4096 2248107 /home/bernardo/sop_lab5
exp01   6714 bernardo rtd   DIR  259,5    4096      2 /
exp01   6714 bernardo txt   REG  259,5   16216 2248566 /home/bernardo/sop_lab5/exp01
exp01   6714 bernardo mem   REG  259,5  1926232 3148231 /usr/lib/x86_64-linux-gnu/libc.so.6
exp01   6714 bernardo mem   REG  259,5   215000 3148224 /usr/lib/x86_64-linux-gnu/ld-linux-x86
-64.so.2
exp01   6714 bernardo 0u   CHR  136,0      0t0      3 /dev/pts/0
exp01   6714 bernardo 1u   CHR  136,0      0t0      3 /dev/pts/0
exp01   6714 bernardo 2u   CHR  136,0      0t0      3 /dev/pts/0
```

Execução do comando `lsof -p`

7. Experimento 7

Ao visualizar a árvore de processos depois de ter executado o código, foi obtido o seguinte resultado:

Figura 11: Elaborada pelo Autor



PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command
11998	bernardo	20	0	35252	884	792	S	0.0	0.0	0:00.00	./threads_vida
11999	bernardo	20	0	35252	884	792	S	0.0	0.0	0:00.00	./threads_vida
12000	bernardo	20	0	35252	884	792	S	0.0	0.0	0:00.00	./threads_vida
12001	bernardo	20	0	35252	884	792	S	0.0	0.0	0:00.00	./threads_vida
12002	bernardo	20	0	35252	884	792	S	0.0	0.0	0:00.00	./threads_vida

Processos no htop

- 1) A cada 30 segundos uma thread filha é encerrada e a linha desaparece da visualização no htop. A linha a ser apagada fica na cor azul.
- 2) Cada thread possui um PID diferente da thread pai.

3) O processo pai é o último a finalizar por causa do uso da função `pthread_join(threads[i], NULL)` dentro do laço de repetição. Como foi o processo pai que fez a chamada, ele suspende a execução da thread principal até que cada thread com índice `i` termine sua tarefa.

4) Ao executar o programa e pressionar `Ctrl+C`, o programa finaliza e todas as threads são encerradas. Isso acontece pois estamos encerrando o processo pai, como as threads filhas compartilham o mesmo espaço de memória, também são encerradas junto com o pai e não se tornam threads orfãs.

8. Experimento 8

1 e 2) Executando o código várias vezes, foi obtido diferentes valores do contador.

Figura 12: Elaborada pelo Autor

```
bernardo@debian:~/sop_lab5$ ./race_condition
Valor final do contador: 1079723 (Esperado: 2000000)
bernardo@debian:~/sop_lab5$ ./race_condition
Valor final do contador: 1007047 (Esperado: 2000000)
bernardo@debian:~/sop_lab5$ ./race_condition
Valor final do contador: 1000000 (Esperado: 2000000)
bernardo@debian:~/sop_lab5$ ./race_condition
Valor final do contador: 1048590 (Esperado: 2000000)
bernardo@debian:~/sop_lab5$ ./race_condition
Valor final do contador: 1008059 (Esperado: 2000000)
bernardo@debian:~/sop_lab5$ ./race_condition
Valor final do contador: 1014693 (Esperado: 2000000)
bernardo@debian:~/sop_lab5$ ./race_condition
Valor final do contador: 1000937 (Esperado: 2000000)
bernardo@debian:~/sop_lab5$ ./race_condition
```

Execução do código `race_condition`

3) O resultado muda pois as duas threads estão rodando simultaneamente dentro do processador, desta forma, cada etapa do contador pode ser sobrescrita por outra thread. Cada vez que o código é executado, o processador pode dar mais tempo para um thread rodar e pode interromper a outra.

4) Executando três vezes o mesmo código, foi observado valores diferentes no tempo de execução.

Figura 13: Elaborada pelo Autor

```
bernardo@debian:~/sop_lab5$ time ./race_condition
Valor final do contador: 1007994 (Esperado: 2000000)

real    0m0,010s
user    0m0,017s
sys     0m0,000s
bernardo@debian:~/sop_lab5$ time ./race_condition
Valor final do contador: 1151466 (Esperado: 2000000)

real    0m0,011s
user    0m0,018s
sys     0m0,000s
bernardo@debian:~/sop_lab5$ time ./race_condition
Valor final do contador: 1053241 (Esperado: 2000000)

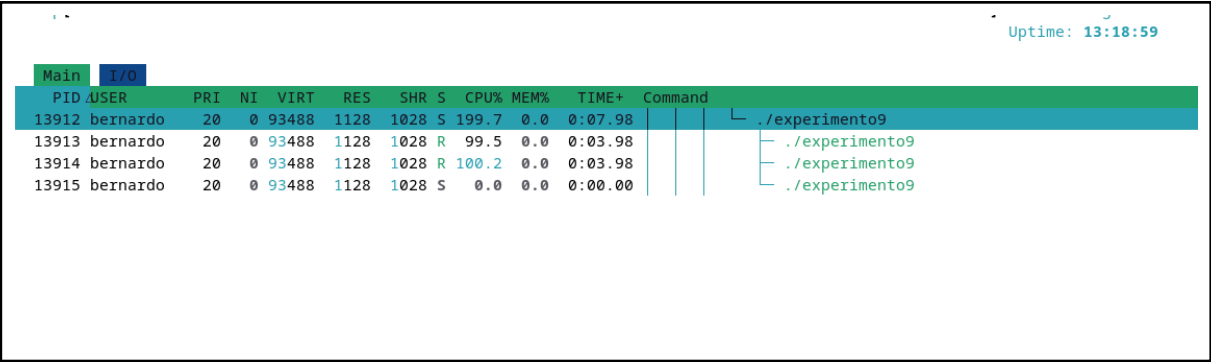
real    0m0,010s
user    0m0,013s
sys     0m0,004s
```

Tempo de execução do código `race_condition`

9. Experimento 9

- Executando o código e abrindo o htop, foi possível observar a seguinte árvore de processos

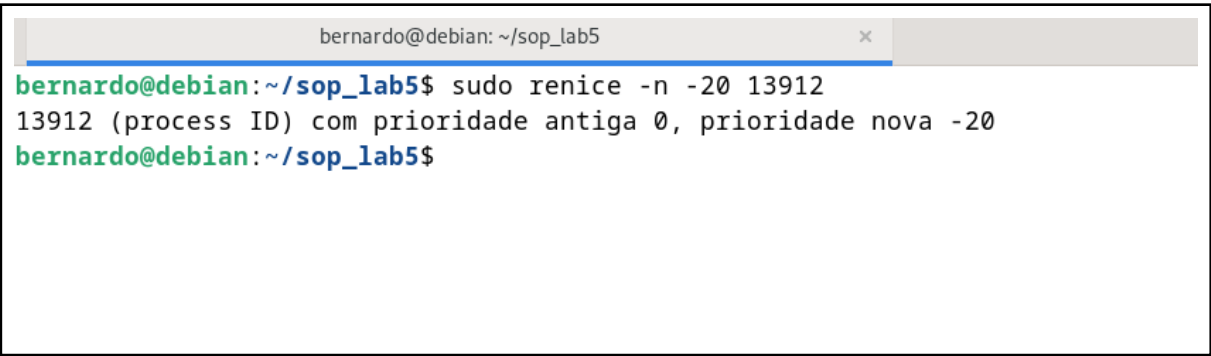
Figura 14: Elaborada pelo Autor



Visualização dos processos no htop

- Foi mudado a prioridade do processo pai de 0 para -20 com o comando renice -n 20 13912

Figura 15: Elaborada pelo Autor



Mudando a prioridade do processo pai usando renice

- No htop foi possível observar o processo pai com a nova prioridade

Figura 16: Elaborada pelo Autor



Nova prioridade do processo pai no htop

10. Experimento 10

- Inicialmente, foi executado o código seguido de um Ctrl+Z e bg para deixar o programa em background.

Figura 17: Elaborada pelo Autor

```
bernardo@debian:~/sop_lab5$ ./exp10
[Worker 0] Aguardando requisicao...
[Worker 1] Aguardando requisicao...
[Worker 2] Aguardando requisicao...
[Worker 3] Aguardando requisicao...
[Worker 4] Aguardando requisicao...
Servidor rodando com 5 Workers. PID: 14359
^Z
[1]+  Parado                  ./exp10
bernardo@debian:~/sop_lab5$ bg
[1]+ ./exp10 &
[Worker 2] Processando...
[Worker 2] Aguardando requisicao...
[Worker 3] Processando...
[Worker 3] Aguardando requisicao...
[Worker 4] Processando...
```

Execução do código

- Para derrubar o servidor, foi inicialmente buscado seu PID em um novo terminal. Posteriormente utilizado o comando `kill -9 [PID]` para encerrar o processo.

Figura 18: Elaborada pelo Autor

```
bernardo@debian:~/sop_lab5$ ps aux | grep exp10
bernardo  14359  0.0  0.0 108984 1116 pts/0    Sl   11:25   0:00 ./exp10
bernardo  14371  0.0  0.0  6356 2208 pts/1    S+   11:26   0:00 grep exp10
bernardo@debian:~/sop_lab5$ kill -9 14359
```

Encerrando o processo do servidor

- No terminal antigo, foi possível observar que o código encerrou e parou sua execução depois do comando `kill`.

Figura 19: Elaborada pelo Autor

```
[Worker 0] Aguardando requisicao...
[Worker 4] Processando...
[Worker 4] Aguardando requisicao...
[Worker 1] Processando...
[Worker 1] Aguardando requisicao...
[Worker 3] Processando...
[Worker 3] Aguardando requisicao...
[Worker 1] Processando...
[Worker 1] Aguardando requisicao...
[Worker 4] Processando...
[Worker 4] Aguardando requisicao...
bernardo@debian:~/sop_lab5$
```

Código parando execução